

SmartHome Control

Introduction

Notre entreprise a été choisie par une grande figure du domaine de la domotique afin de concevoir et développer un système d'automatisation pour maisons connectées.

Le système doit permettre aux utilisateurs de gérer et contrôler leurs équipement domestique et d'automatiser certaines tâches à partir d'une interface simple et accessible.

Le projet présente des enjeux importants :

- Sécurité des données et équipements
- Fiabilité des commandes
- Disponibilité du système
- Simplicité d'utilisation
- Évolutivité et maintenance
- Traçabilité des opérations
- Respect des exigences en matière de données personnelles

Après analyse du contexte, le Cycle en V est retenu comme méthodologie de développement.

Objectifs du Projet

SmartHome Control s'articule autour de ces fonctionnalités clés :

- Gestion des utilisateurs consultation de leur statuts
- Gestion de un ou plusieurs foyer et consultation de leur état
- Gestion des équipements consultation de leur état
- Contrôle des équipement à distance
- Planification des tâches
- Recevoir des notifications
- Consulter un historique des actions effectués
- Gestions des dysfonctionnement et des erreurs

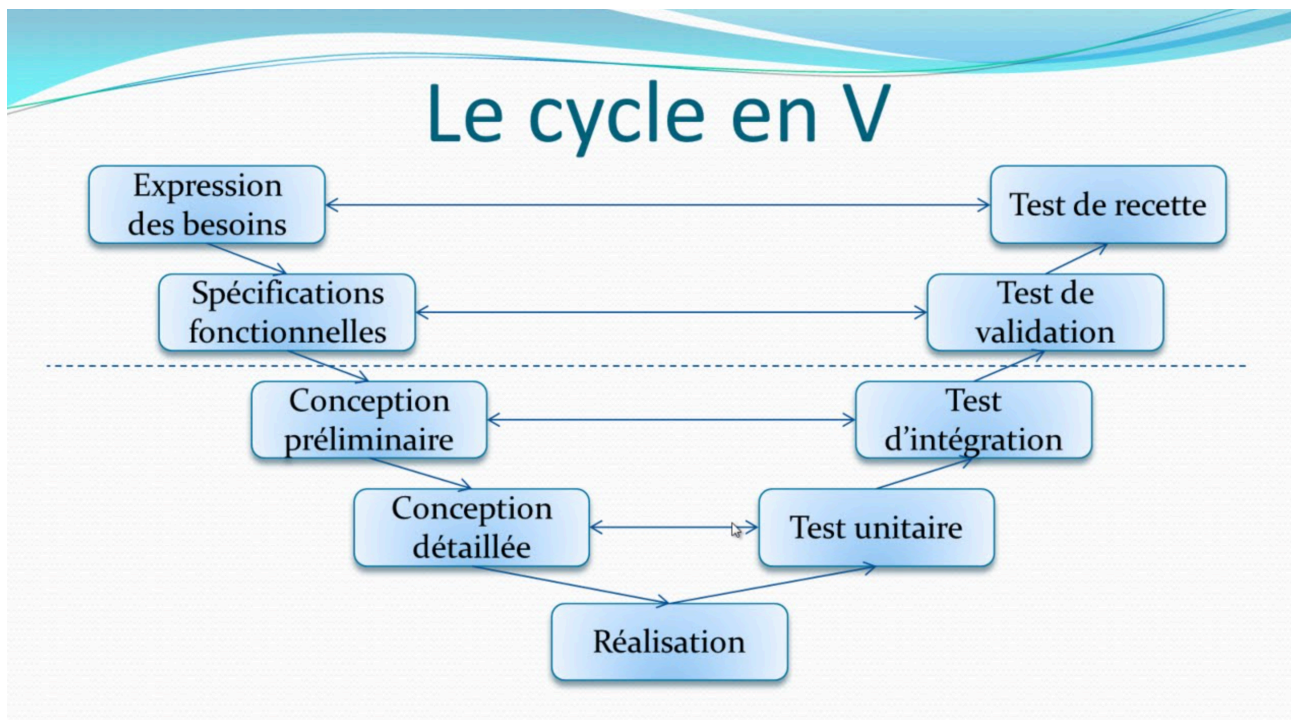
Un utilisateur peu créer une règle qui entraîne une action :

Si la température descend en dessous de 19°C entre 20h et 06h, allumer le chauffage à 21°C.

Le système devra alors gérer :

condition —> déclencheur —> action —> confirmation —> notification —> historique

Pourquoi choisir le Cycle en V ?



Le Cycle en V, en gestion de projet, est une méthodologie de travail qui découle du modèle Waterfall permettant de représenter des processus de développement de manière linéaire et en phases successives. Mais il s'agit avant tout d'ingénierie système. Ce modèle a été développé dans les années 1980 et appliqué au champs des projets industriels puis étendu aux projets informatiques. Le cycle en V apporte une réponse au problème des tests improvisés et livrables qui ne répondent plus au cahier des charges initial et aux retards évitables.

Cependant aujourd'hui, il est plus efficace sur les projets dont le cahier des charges inchangé du début à la fin, dont l'environnement technique évolue très peu, dont la documentation est exhaustive à chaque phase et pour des projets simple avec des objectifs clairs et des délais courts.

La lettre V fait référence à la vision schématique de ce cycle qui se constitue d'une phase descendante puis d'une phase ascendante. Il associe à chaque phase de réalisation une phase de validation. Chaque phase descendante trouve son miroir exact côté validation. Donc la phase de gauche définit ce que le système doit faire et comment il sera construit et la phase de droite vérifie que le système construit correspond bien à ce qui avait été défini.

Nous avons donc trois grands axes :

Phase descendante	=	Conception
Pointe du V	=	Mise en oeuvre et développement
Phase ascendante	=	Vérification validation

Phase descendante :

La phase descendante concerne tout ce qui est lié à la conception du projet. Il s'agit de la phase de conception pendant laquelle les besoins sont définis grossièrement et détaillés de plus en plus précisément.

Analyse et expression du besoin :

La première étape de cette phase est l'exigence ou les expressions du besoins. Il est nécessaire et même crucial que la structure et les exigences du projet soient claires et sans ambiguïté. Cette étape doit porter sur des éléments nécessaires, rédigés dans un langage simple à lire, cohérents, réunis au même endroit, précis et incluant une marge d'erreur, accessibles, et vérifiables via une inspection, une analyse, une démonstration ou un test. C'est ici que l'on va rassembler le plus d'informations possible. Les parties suivantes vont dépendre de cette étape cruciale. On va donc y investir beaucoup de temps. Nous allons y retrouver le cahier des charges avec : l'objectif du projet, les besoins clients, le public cible, le MVP, les contraintes fonctionnelles, les moyens techniques, les délais, le budget et le plan de management du projet (organisation de l'équipe), la validation des tests, quels sont les risques et comment les gérer ainsi que le plan de recette (critères d'acceptation). Cette étape est sous la responsabilité de la maîtrise d'ouvrage.

Les spécifications fonctionnelles :

Ici, on entre dans la partie analyse des spécifications fonctionnelles du projet. On obtiendra donc le cahier des charges fonctionnel qui contiendra les règles de gestion métier, les user stories, les wireframes et maquettes et la planification des tâches, éventuellement des diagrammes de cas d'utilisation, de séquence ou le modèle de conception de base de données (MCD) et le plan de validation. Une stratégie de tests fonctionnels ou end-to-end (en fonction des moyens et des délais) sera rédigée. Cette étape est sous la responsabilité de la maîtrise d'œuvre.

Conception générale :

Dans cette partie nous allons aborder les points suivants. L'infrastructure va définir le matériel utilisé et les pratiques nécessaires pour stocker et mettre en ligne l'application et ses données, notamment avec un diagramme de déploiement. Puis nous nous intéresserons aux choix technologiques pour définir quels langages et packages seront choisis pour coder l'application. La suite concerne le schéma de modèle logique de bases de données (MLD) qui apportera plus de précision sur comment les données de l'application seront organisées et liées entre elles. Ensuite, on a l'architecture logicielle. Nous expliquerons comment les différentes parties de l'application seront organisées et reliées entre elles. Enfin, on termine cette partie par les contraintes techniques qui serviront à déterminer les niveaux d'exigence au sujet de la sécurité, la compatibilité ou des performances de l'application et l'ensemble des échanges entre le système et les services externes. Un plan de tests d'intégration sera rédigé à cette étape.

Conception détaillée :

La quatrième et dernière étape de la phase descendante est la conception détaillée avant la phase de développement de l'application. On spécifiera davantage la structure interne de l'application et donc de chaque module, chaque composant, la gestion des exceptions et la structure des classes et fonctions. Pour cela on pourra réaliser des diagrammes de composants, des diagrammes d'états, et des diagrammes de classes. Nous entrerons ensuite en détails dans la structure de la base de données avec le modèle physique de la base de données (MPD) qui contiendra les tables, les champs, les relations, les types, et les contraintes. On rédigera les tickets utilisés en phase de développement. Un plan de tests unitaire sera rédigé pour s'assurer du bon fonctionnement individuel de toutes les parties de l'application. On détermine aussi comment la maintenance et la mise à jour est définie.

Point important :

Il est très important que le client valide l'expression du besoin et la partie spécification technique.

Dans le cas où cette validation n'est pas strictement effectuée, on prends le risque de partir sur de mauvaises bases. À cause de cela, on s'expose à des failles et des risques de reprise structurelles qui ne sont pas adaptés au cycle en V mais plus aux méthodes agiles.

Phase de développement (mise en œuvre) :

La pointe du V représente la phase de développement du projet. Les développeurs vont prendre les tickets, qui sont basés sur les phases précédentes et les exécuter.

Toute l'application, front-end et back-end, est développée dans cette étape. Les retours clients sont donc bien moins fréquents que dans la méthode agile, d'où l'intérêt d'être exhaustif et précis lors de la phase descendante.

Phase ascendante :

Cette branche de droite regroupe toutes les étapes de validations et de tests définies dans la phase de gauche et se conclue par la mise en ligne de l'application.

Tests unitaires :

En se basant sur le plan de tests unitaires, on va exécuter les tests unitaires de l'application. L'objectif est de vérifier chaque sous composant et fonction de l'application de manière isolée.

Tests d'intégration :

On vérifie que les différentes parties de l'application communiquent correctement entre elles et que le système communique avec les services externes de l'application, le tout sans créer de conflit.

Tests systèmes (ou validation) :

Les tests fonctionnels ou end-to-end sont exécutés par la maîtrise d'œuvre afin de vérifier que le système complet fonctionne et respecte les spécificités techniques et fonctionnelles. On lance aussi les tests de sécurité, de performance, de charge, de récupération et de non régression. Ces tests sont réalisés dans un environnement proche de la production, idéalement la pré-production.

Tests de recette (ou acceptation) :

Validation finale par le client ou par la maîtrise d'ouvrage pour confirmer que le besoin exprimé au début du projet est rempli.

Mise en service :

Une fois toutes les étapes passées, le projet est déployé.

Avantages :

C'est une méthode simple à mettre en oeuvre si les besoins et les exigences sont clairement définis. S'ils sont exprimés avant la phase de développement, alors on limite les différentes interprétations différentes entre client, chef de projet, développeurs, UX/UI, QA, et les architectes.

On se retrouve alors avec des objectifs clairs et des tests de contrôle soit réfléchis soit exécutés à chaque phase. Cela permet de détecter des incohérences avant même que le code ne soit développé. Les différentes phase de tests garantissent un produit final avec le moins d'erreurs possibles voir aucune.

La Traçabilité est assurée avec chaque exigence qui est associée à une documentation notamment au niveau de l'architecture aux composants, aux fonctionnalités et aux tests.

Le cycle en V permet aussi une meilleur visibilité pour la direction puisque le projet est découpé en jalons. Le PDG dispose ainsi d'une visibilité sur l'avancement du projet.

Limites et erreurs courantes:

Le cycle en V présente également des limites, notamment une difficulté à gérer les changements. Modifier une exigence après la phase de conception peu avoir des conséquences importantes et entraîner des difficultés en cascade. Les changements doivent être maîtrisés et arrivés plutôt au début de la phase descendante.

Sous estimer la phase d'analyse des besoins (cahier des charges incomplet = reprise en cascade, spécification fonctionnelles floues) est également une erreur. Cette étape doit être faite de manière rigoureuse et précis pour éviter tout blocage ensuite.

Ignorer l'effet tunnel, pas de retour avec les parties prenantes entre conception et tests, présentes aussi des risque de voir le produit conforme aux spécifications mais pas aux attentes réelles du client.

Si plus de 20% des exigences ont été modifier entre la mi-projet ce n'est plus la bonne méthode. De plus, Valide une phase sous pression de planning dans réelle vérification n'est pas une bonne façon de faire et peu entraîner des difficultés aux ramifications complexes.

La détection tardive des problèmes doit se faire dès la phase descendante pour qu'ils soient identifiés et corrigés le plus tôt possible.

Enfin, la méthode est particulièrement inadapté aux projets longs, complexes et modulables.

Palier aux erreurs et inconvénients :

Pour pallier aux limites du cycle en V, plusieurs moyens peuvent être mis en place.

Par exemple, on peut utiliser des approches et des itérations courtes pour des cycles rapide et avoir un retour client rapidement, ce qui permet d'avoir des livrables sous forme de version, cela permet d'avoir la présence du prestataire qui est impliqué l'ensemble des étapes.

En cas de changement des exigences, une analyse doit être faite et validée afin de le mettre en place de façon efficace.

Toute modification importante devra être analysée selon les impacts fonctionnels, techniques, de sécurité, de planning, budgétaires, et des tests. Le changement devra être validé par les responsables concernés.

Pourquoi le cycle en V est adapté au Projet SmartHome :

Sécurité :

Une plateforme domotique peut contrôler des équipements physiques présents dans le domicile. Une faille peut avoir des conséquences importantes. Par exemple, un accès non autorisé au système prive l'utilisateur des toutes les fonctionnalités automatiques et à distance. Un équipement peut s'activer ou se désactiver sans intervention de l'utilisateur. L'alarme de sécurité peut être désactivée. Si une personne malveillante peut accéder aux données personnelles des utilisateurs et consulter les habitudes des occupants.

Il est donc nécessaire de définir les exigences de sécurité avant l'implémentation et de prévoir leur validation.

Interaction avec des équipement physiques :

Le système ne se limite pas à une application. Il doit communiquer avec des capteurs, des éclairages, des thermostats, des volets, des systèmes de sécurité et une passerelle domotique. Les interfaces entre ces composants doivent être précisément définies. Le Cycle en V permet de formaliser ces interfaces avant la phase d'intégration.

Besoin de traçabilité :

Pour chaque exigence importante, nous devons pouvoir répondre à la question :
Comment avons nous vérifié que cette exigence respectée ?

Le cycle en V permet de construire une chaîne de traçabilité :

exigence -> conception -> implémentation -> tests -> résultat (produit)

Cette traçabilité est d'autant plus intéressante pour les exigences de sécurité, de performance et de disponibilité.

Conclusion et synthèse

Phase 1 : Analyse des exigences

- Cahier des charges
- Plan de recette (critères d'acceptation)
- Registre initial des risques

Phase 2 : Spécifications fonctionnelles

- Cahier des charges fonctionnel
- Wireframes
- Maquettes
- Planification des tâches
- Diagrammes de cas d'utilisation
- Diagrammes de séquence
- MCD
- Stratégie de tests fonctionnels ou end-to-end
- Plan de validation

Phase 3 : Conception générale

- Infrastructure et diagramme de déploiement
- Benchmark technologique
- MLD
- Architecture logicielle
- Contraintes techniques
- Stratégie des tests d'intégration

Phase 4 : Conception détaillée

- Diagrammes composants
- Diagrammes d'état
- Diagrammes de classes
- MPD
- Contrats API
- Règles métier
- Stratégie de maintenance
- Plan de tests unitaires

Phase 5 : Mise en Œuvre et développement

- Code source
- Documentation technique
- Tests unitaires
- Documentation API (Swagger)

Phase 6 : Tests unitaires

- Lancement de chaque test et rapport de tests unitaires à partir du plan de tests (Phase 4)

Phase 7 : Tests d'intégration

- Lancement de chaque test et rapport de tests d'intégration à partir de la stratégie de tests (Phase 3)

Phase 8 : Tests système (Tests de validation)

- Lancement de chaque test et rapport de tests fonctionnel ou end-to-end à partir de la stratégie de tests (Phase 2)

Phase 9 : Tests de recette (Tests d'acceptation)

- Vérification du produit par le client

Phase 10 : Mise en service

- Le projet est déployé

